QuillAudits

AUDIT REPORT

October 2026

For



Tajir Chain

Table of Content

Executive Summary 04

Number of Security Issues per Severity 09

Summary of Issues 10

Checked Vulnerabilities 11

Techniques and Methods 12

Types of Severity 13

Types of Issues 14

Severity Matrix 15

- Impact 15
- Likelihood 15

High Severity Issues 16

- 1. Unprotected inherited initializer allows seizure of an uninitialized vesting wallet 16
- 2. A single global revoked flag voids the vesting schedule for every asset held 19

Medium Severity Issues 22

- 3. renounceOwnership() and transferOwnership() are inherited un-overridden 22
- 4. Unguarded subtraction underflows against non-conserving tokens (rebasing or fee-on-transfer) 24
- 5. initialize() never validates startTimestamp 26
- 6. Clawback commits an irreversible flag with no balance-delta assertion 28
- 7. Schedule parameters are read live with no clamp 30

Low Severity Issues 32

- 8. Tranche quantization lets a revocation strip a nearly-elapsed tranche 32
- 9. TajirToken.initialize silently rescales initialSupply 33

Informational Severity Issues 35

10. revoked has no getter and its guard precedes the authorization gate 35

11. withdrawUnreleased re-derives releasable() inline 36

12. Floor division routes rounding remainders away from the beneficiary 37

13. upgradeToAndCall inherited payable with no ETH-handling path 38

Functional Tests 39

Threat Model 42

1. External Dependencies & Trust Boundaries 42

2. Entry & Exit Points Analysis (Function-Level) 45

3. Asset Flow Mapping (Critical Contracts) 51

Fuzzing Results 53

Automated Tests 55

Closing Summary & Disclaimer 56

Executive Summary

Version 2.0 (revised final) | Issued 16 October 2026 | Supersedes the earlier October 2026 report, including Review 4 | Client: Tajir Protocol Ltd.

Project Name	Tajir Chain
Protocol Type	Vesting, ERC20
Project URL	https://www.tajirchain.com/
Overview	Tajir Chain's token and vesting layer. TajirToken (TJR) is an ERC-20 with EIP-2612 permit on Ethereum mainnet, deployed behind an existing ERC-1967 proxy (TajirTokenProxy). An initial supply of 650,000,000 TJR was minted once at initialization; this initial supply is distinct from circulating or current total supply. The reviewed code provides no further minting, and supply can be reduced only through the reviewed burn functions (burn and burnFrom). In the Review 4 code, `_authorizeUpgrade` reverts for every caller. The Review 4 implementation was applied to the live proxy on 16 October 2026 (transaction `0xbb10e7ba9158d2582e76c21627416290722b07348bc00c86dd69265f238fcdbf`), so implementation upgrades of the live token are now permanently disabled. The existing ERC-1967 proxy remains and the token address is unchanged. TJR is an ERC-20 on Ethereum mainnet. TJR deposited through the bridge is locked on Ethereum and represented as native TJR on Tajir Chain; the bridge implementation was outside the scope of this review, so this description follows project documentation. TajirVestingWallet extends OpenZeppelin's VestingWalletCliff with tranche-based release: nothing unlocks until the cliff, after which the allocation vests in N equal installments. Each wallet vests a single asset fixed at initialization and provides a one-shot admin clawback of the unvested portion of that asset. The vesting wallets cover only the allocations deployed into them.
Audit Scope	The scope of this Audit was to analyze the TajirChain **Smart Contracts** for quality, security, and correctness.
Source Code link	Public source reference: public commit https://github.com/Tajir-Chain/core-contracts/commit/8a27cb81bce64d82836a9bf89c3c81626dac5f62 (16 October 2026) of https://github.com/Tajir-Chain/core-contracts. The audit was performed on the project's private repository, which remains private because it contains upcoming contracts outside the scope of this audit.

Source Code link *(continued)*	Mapping of each in-scope file, verified by file comparison and by compiling the public files with the audited build settings: (1) `TajirToken.sol` : https://github.com/Tajir-Chain/core-contracts/blob/8a27cb81bce64d82836a9bf89c3c81626dac5f62/src/token/TajirToken.sol corresponds to the audited commit `94551139e0cb1f03f97bdecf1344003331f7182a`. (2) `TajirTokenProxy.sol` : https://github.com/Tajir-Chain/core-contracts/blob/8a27cb81bce64d82836a9bf89c3c81626dac5f62/src/token/TajirTokenProxy.sol corresponds to the audited commit `94551139e0cb1f03f97bdecf1344003331f7182a`. (3) `TajirVestingWallet.sol` : https://github.com/Tajir-Chain/core-contracts/blob/8a27cb81bce64d82836a9bf89c3c81626dac5f62/src/TajirVestingWallet.sol corresponds to the final reviewed vesting commit `b04f613864d463d88963630c25fbde68187cb9ca`. It is also identical to the Etherscan-verified source of the TajirVestingWallet implementation deployed on Ethereum mainnet at `0xa77b3d915c98df7eaa5636ddb8f64c53f517112a`. The earlier public commit `69aad7510910c48f1c5e193a2f5add84819dc630` contained the pre-fix vesting version. For all three files, the public copies have the same content as the audited versions: they use CRLF line endings, are byte-identical once normalised to LF, and compile to bytecode identical to the audited build.
Contracts in Scope	TajirVestingWallet.sol, TajirToken.sol, TajirTokenProxy.sol
Branch	foundry
Commit Hash	`01802d8370699dbe09ec1113df59c6388c32eb51`
Final Reviewed Commits	TajirToken.sol and TajirTokenProxy.sol: `94551139e0cb1f03f97bdecf1344003331f7182a` (Review 4). TajirVestingWallet.sol: `b04f613864d463d88963630c25fbde68187cb9ca` (logic identical to the SOW 1 final vesting review commit `9b174a414604366232f8aa66737c75fcd1808b84`; unchanged in `94551139e0cb1f03f97bdecf1344003331f7182a`).
Language	Solidity (compiler 0.8.27, optimizer enabled with 200 runs, EVM version cancun). Dependencies: OpenZeppelin Contracts and OpenZeppelin Contracts Upgradeable v5.5.0.
Blockchain	`TajirToken` and `TajirTokenProxy` : **Ethereum mainnet (Chain ID 1)**. `TajirVestingWallet` : implementations deployed on Ethereum mainnet (Chain ID 1) and Tajir Chain mainnet (Chain ID 3377). Vesting proxies for the main allocations are **planned** on Ethereum mainnet, holding ERC-20 TJR directly.
Method	Manual Analysis, Functional Testing, Fuzzing, Automated Testing

Review 1	21st July 2026
Updated Code Received	24th July 2026
Review 2	27th July 2026
Fixed In	`f70f18049eeb85102bbacf947942912cacec107f`
Review 3	On 27th August 2026, the Tajir team made changes to the token contract post-audit and requested a quick review. The changes add holder self-burn through `burn(uint256)` and allowance-authorised burn through `burnFrom(address,uint256)`, with no privileged administrative burn (standard ERC20Burnable behaviour; an approved spender may call burnFrom). The QuillAudits team reviewed the changes and found no new vulnerabilities. Commit: `c6c860315e7de8b9e0adb6cb13def170b4630bfd` (private repository).
SOW 1 Final Vesting Review	Date: 10th September 2026. Scope: TajirVestingWallet.sol (fix review and re-audit of Issues 1–13). This review is separate from the token burn review labelled Review 3. Reviewed commit: `9b174a414604366232f8aa66737c75fcd1808b84` (fixes introduced in `34a43fce294609f87365d544f4515f148ee1bd59`). Final vesting source: `b04f613864d463d88963630c25fbde68187cb9ca` (adds a code comment only; logic unchanged). Findings, documented in the separate report TajirVestingWallet — Patch Review & Re-Audit (10 September 2026)
Review 4	On 14th October 2026, the Tajir team changed the token contract to disable upgradeability, and a post-audit review was conducted scoping the token only. In the reviewed code the inherited upgradeToAndCall entry point remains, while `_authorizeUpgrade` reverts for every caller. We confirmed that the reviewed code matches this description and that no other callable function can change the implementation. Only src/token/TajirToken.sol (and its unit test) changed in this commit; the other in-scope files did not change. The QuillAudits team reviewed the changes and found no new vulnerabilities. Commit: `94551139e0cb1f03f97bdecf1344003331f7182a` (private repository; public reference above).
Review 4 Deployment Evidence	Chronology (UTC): the Review 4 implementation `0xa1F5F61Ae9C55b16Cf9FeAC6b499b00C563de6c5` was deployed on 15 October 2026 at 16:29:11 (Ethereum block 26,199,117). On 16 October 2026, pre-execution bytecode verification was completed at 08:40:11 (block 26,203,957). The bytecode confirmation therefore preceded execution.

Review 4 Deployment Evidence *(continued)*	(a) Pre-execution bytecode confirmation: the runtime bytecode of `0xa1F5F61Ae9C55b16Cf9FeAC6b499b00C563de6c5` matches `src/token/TajirToken.sol` at commit `94551139e0cb1f03f97bdecf1344003331f7182a` compiled with the audited build settings (solc 0.8.27, optimizer 200 runs, cancun, OpenZeppelin v5.5.0), including the metadata hash. The only differing bytes are the compiler-generated immutable slots, which hold the implementation's own address. The previous implementation `0x48bf1A70E9509B59831Dd23921eB9570F1ED7BA8` was verified from the proxy's ERC-1967 slot; its executable bytecode matches the Review 3 commit `c6c860315e7de8b9e0adb6cb13def170b4630bfd`, and it is the storage-layout baseline. Storage layout is identical between Review 3 and Review 4. (b) Pre-execution fork simulation (local fork of Ethereum mainnet at block 26,203,957; not a live transaction): the planned Safe call wrote only the implementation slot. Name, symbol, decimals, total supply, sampled balances, permit nonces and domain, and roles were unchanged, and further upgrade attempts failed. The project's Tenderly simulation of the same Safe transaction (Safe nonce 16) was also checked: the calldata, Safe transaction hash `0x2d3d54525655a79f8157c754e0190d5654839c113edd12a22df5a55696f9d0a8` and domain hash matched. (c) Post-upgrade verification on Ethereum mainnet (live chain): transaction `0xbb10e7ba9158d2582e76c21627416290722b07348bc00c86dd69265f238fcdbf` (block 26,204,693, 16 October 2026 11:07:47 UTC) was executed by the token Safe `0xBA1D51BBB17Fd24Fd6bF2c030F3ca0E3a3B4d22E` with 3 of 5 owner signatures. It called `upgradeToAndCall(0xa1F5F61Ae9C55b16Cf9FeAC6b499b00C563de6c5, 0x)` on proxy `0x9D98C61d1136cfA2ac263Be355350C97Ca41c110` with zero value, and emitted `Upgraded(0xa1F5F61Ae9C55b16Cf9FeAC6b499b00C563de6c5)` and `ExecutionSuccess(0x2d3d…d0a8)`. Results: the implementation slot now holds `0xa1F5F61Ae9C55b16Cf9FeAC6b499b00C563de6c5`, and its bytecode is unchanged from the verified build. A trace of the transaction shows that the only token storage write was the implementation slot; the only other write was the Safe nonce, 16 to 17. Comparing block 26,204,692 (before) with block 26,204,693 (after), the following were identical: name "Tajir", symbol "TJR", decimals 18, total supply 650,000,000 TJR, DOMAIN_SEPARATOR and eip712Domain, both role assignments, the balances of 34 sampled accounts (5 holders and 29 allowance owners), 30 sampled allowances (29 non-zero), and the permit nonces of the 29 allowance owners. No RoleGranted or RoleRevoked event has occurred since deployment. Subsequent upgrade attempts against the live state are rejected: from the Safe with Not Upgradeable Anymore! (also when sending ETH), from any other address with AccessControlUnauthorizedAccount, and initialize with InvalidInitialization.
Vesting Deployment and Governance	Planned controls (not yet deployed or verified): vesting proxies for the main allocations on Ethereum mainnet holding ERC-20 TJR directly (vestedAsset = TJR proxy `0x9D98C61d1136cfA2ac263Be355350C97Ca41c110`).

Vesting Deployment and Governance *(continued)*	For each proxy, admin and upgrader are to be two distinct addresses, each subject to an enforced 48-hour delay, with the Veto Safe able to cancel queued actions. These remain planned controls until their addresses and configuration are supplied and verified. Ethereum vesting coverage: the planned Ethereum mainnet vesting of ERC-20 TJR uses the audited ERC-20 release path release(address) and the audited clawback path withdrawUnreleased(token, to) with token equal to the wallet's vestedAsset, as implemented in TajirVestingWallet.sol at commit `b04f613864d463d88963630c25fbde68187cb9ca`. The TajirVestingWallet implementation deployed on Ethereum mainnet at `0xa77b3d915c98df7eaa5636ddb8f64c53f517112a` has executable bytecode matching that source; the implementation on Tajir Chain mainnet (Chain ID 3377) at `0x8284bd32d15df0c46c05590711779db9890045fa` also matches.

✅ Verify the Authenticity of Report on QuillAudits Leaderboard: https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	0 (0%)
High	2 (15%)
Medium	5 (38%)
Low	2 (15%)
Informational	4 (31%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	**2**	0
Partially Resolved	0	0	0	0	0
Resolved	0	**2**	**5**	0	**4**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Unprotected inherited initializer allows seizure of an uninitialized vesting wallet	High	Resolved
2	A single global revoked flag voids the vesting schedule for every asset held	High	Resolved
3	renounceOwnership() and transferOwnership() are inherited un-overridden	Medium	Resolved
4	Unguarded subtraction underflows against non-conserving tokens (rebasing or fee-on-transfer)	Medium	Resolved
5	initialize() never validates startTimestamp	Medium	Resolved
6	Clawback commits an irreversible flag with no balance-delta assertion	Medium	Resolved
7	Schedule parameters are read live with no clamp	Medium	Resolved
8	Tranche quantization lets a revocation strip a nearly-elapsed tranche	Low	Acknowledged
9	TajirToken.initialize silently rescales initialSupply	Low	Acknowledged
10	revoked has no getter and its guard precedes the authorization gate	Informational	Resolved
11	withdrawUnreleased re-derives releasable() inline	Informational	Resolved
12	Floor division routes rounding remainders away from the beneficiary	Informational	Resolved
13	upgradeToAndCall inherited payable with no ETH-handling path	Informational	Resolved

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls
- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments, match logic and expected behavior.
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper/project documentation.
- Implementation of ERC standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper/documentation. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

High Severity Issues

HIGH · ISSUE 1

Unprotected inherited initializer allows seizure of an uninitialized vesting wallet

Resolved

Path
src/TajirVestingWallet.sol

Function Name
`initialize(address,uint64,uint64)` (inherited), `initialize(...)` (8-arg, L82-111)

Description

`TajirVestingWallet` inherits `VestingWalletUpgradeable.initialize(address,uint64,uint64)`, which is declared `public virtual initializer` at `VestingWalletUpgradeable.sol:57`. The contract declares its own 8-argument `initialize(...)` at with a **different function selector**, and does **not** mark it `override`. Both functions therefore remain externally callable on the deployed contract. This was confirmed against the compiled artifact:

```
$ forge inspect src/TajirVestingWallet.sol:TajirVestingWallet methods
| initialize(address,uint64,uint64)                                | 989a8366 |
| initialize(address,uint64,uint64,uint64,uint64,uint64,address,address) | ad562987 |
```

Both paths consume the same one-shot `initializer` flag, so whichever executes first wins. `_disableInitializers()` in the constructor protects only the implementation contract, it never touches the proxy's storage.

The window is reachable because deployment is **not atomic**. `script/DeployVesting.s.sol` constructs the proxy with empty init calldata and initializes in a separate call at L172:

```
1  TajirVestingWallet impl = new TajirVestingWallet();
2  ERC1967Proxy proxy = new ERC1967Proxy(address(impl), "");   // L169  empty init data
3  wallet = TajirVestingWallet(payable(address(proxy)));
4  wallet.initialize(                                          // L172  separate transaction
5      cfg.beneficiary, startTimestamp, cfg.duration, cfg.cliff,
6      cfg.trancheDuration, cfg.totalTranches, admin, upgrader
7  );
```

Impact Medium

An attacker who lands the 3-argument path first becomes `owner()` the beneficiary who receives every `release()` and consumes the `initializer` flag. The deployer's 8-argument call then reverts with `InvalidInitialization`, failing the deployment script.

HIGH · ISSUE 1

Unprotected inherited initializer allows seizure of an uninitialized vesting wallet *(continued)*

Resolved

Because `DeployVesting.s.sol` performs **no funding** (there is no `transfer` or `mint` anywhere in the script), the attacker seizes an **empty** wallet. The realised harm is therefore a repeatable **denial of service on deployment** plus wasted gas, rather than direct theft. Every redeployment attempt is equally front-runnable, so an attacker can block vesting deployment indefinitely.

Direct fund loss requires a compounding operational error funding a wallet whose initialization reverted. Should that occur, the seized wallet is unrecoverable: `admin` and `upgrader` are `address(0)` (no clawback, no upgrade path) and `trancheDuration == 0` makes `_vestingSchedule` divide by zero at, bricking every value-exit path.

Likelihood High

Permissionless, requires only mempool observation, and no capital.

POC

```
function test_shadowInitializerSeizure() public {
    ERC1967Proxy proxy = new ERC1967Proxy(address(impl), ""); // uninitialized
    OriginalTajirVestingWallet w = OriginalTajirVestingWallet(payable(address(proxy)));

    vm.prank(attacker);
    VestingWalletUpgradeable(payable(address(w))).initialize(attacker, uint64(block.timestamp), 360 days);

    assertEq(w.owner(), attacker);      // attacker is now beneficiary
    assertEq(w.trancheDuration(), 0);   // schedule params never set

    // the legitimate deployment call now reverts with InvalidInitialization
    vm.expectRevert();
    w.initialize(beneficiary, uint64(block.timestamp), 360 days, 30 days, 30 days, 11, admin, upgrader);
}
```

Recommendation

Apply both fixes

1. Override and permanently disable the inherited initializer:

```
function initialize(address, uint64, uint64) public pure override {
    revert("TajirVestingWallet: use the 8-argument initialize");
}
```

2. Make deployment atomic by passing the initialization calldata as the proxy constructor's `data` argument, removing the window entirely:

HIGH · ISSUE 1

Unprotected inherited initializer allows seizure of an uninitialized vesting wallet *(continued)*

Resolved

```
bytes memory initData = abi.encodeCall(
    TajirVestingWallet.initialize,
    (cfg.beneficiary, startTimestamp, cfg.duration, cfg.cliff,
     cfg.trancheDuration, cfg.totalTranches, admin, upgrader)
);
ERC1967Proxy proxy = new ERC1967Proxy(address(impl), initData);
```

Note

The inherited initializer is disabled in code. Atomic initialization is enforced by the deployment script (DeployVesting.s.sol passes the initialization data to the proxy constructor) rather than by the contract, because the 9-argument initialize remains unrestricted on an uninitialized proxy. This is an accepted design choice, not an unresolved fix; vesting proxies must be deployed with initialization data in the proxy constructor.

High Severity Issues

HIGH · ISSUE 2

A single global revoked flag voids the vesting schedule for every asset held

Resolved

Path
src/TajirVestingWallet.sol

Function Name
`withdrawUnreleased(address,address payable)`, `_vestingSchedule(uint256,uint64)`

Description

The vesting wallet stores revocation as a **single global boolean** (`bool revoked`, L35) while allocations are derived **per asset** from live balances.

`withdrawUnreleased(token, to)` accepts an admin-chosen `token` address and writes that one flag at

```
1  function withdrawUnreleased(address token, address payable to) external {
2      TajirVestingWalletStorage storage $ = _getTajirVestingWalletStorage();
3      if ($.revoked) revert AlreadyRevoked();
4      if (msg.sender != $.admin) revert NotAdmin();      // L122  admin-gated
5      ...
6      uint256 claimable = vested - alreadyReleased;      // L131
7      if (currentBalance <= claimable) revert NothingToWithdraw();
8      uint256 withdrawable = currentBalance - claimable;
9      $.revoked = true;                                  // L137  global, irreversible
10     ...
11 }
```

`_vestingSchedule` consults that same flag for **every** asset:

```
1  } else if ($.revoked || timestamp >= end()) {
2      return totalAllocation;                            // L177-178  fully vested
3  }
```

Revoking against asset X therefore routes asset Y straight to fully-vested, regardless of elapsed time, cliff, or tranche count. There is no un-set path `AlreadyRevoked` at L121 makes it permanent.

The dangerous path requires no malice. An admin sweeping an unwanted or airdropped token out of the wallet is performing an ordinary operation. That one call fully vests the entire remaining allocation and permanently burns the one-shot clawback for every other asset.

HIGH · ISSUE 2

A single global revoked flag voids the vesting schedule for every asset held *(continued)*

Resolved

A secondary path exists but requires role overlap. Because `claimable` and `currentBalance` are both read from the admin-supplied `token`, a token that merely *reports* a large `balanceOf` and no-ops its `transfer` satisfies the `currentBalance > claimable` guard at while moving nothing defeating the documented invariant that `revoked` cannot be set without moving a non-zero surplus. An honest admin has no incentive to do this, since it benefits the beneficiary at the protocol's expense; it becomes exploitable only where `admin` and `beneficiary` are the same key.

Impact High

A single admin action irreversibly fully-vests every asset the wallet holds and permanently destroys the clawback right. The beneficiary can then `release()` 100% of the balance ahead of the intended cliff and tranche schedule. The economic guarantee of the vesting schedule is lost for the entire remaining term.

A related consequence was separately confirmed: because the revoked branch recomputes `totalAllocation` live, every **post-revocation** deposit is also 100% releasable on arrival.

Likelihood Medium

Requires an admin transaction, but the triggering action sweeping an unwanted asset is routine and the destructive side effect is entirely unsignalled in the code.

POC

```
function test_revokeOnJunkTokenFullyVestsEverythingElse() public {
    // wallet holds the real allocation, still pre-cliff
    deal(address(tjr), address(wallet), 1_000_000e18);
    assertEq(wallet.vestedAmount(address(tjr), uint64(block.timestamp)), 0);

    // an unrelated junk token is airdropped in, admin sweeps it
    MockERC20 junk = new MockERC20();
    junk.mint(address(wallet), 1e18);

    vm.prank(admin);
    wallet.withdrawUnreleased(address(junk), payable(treasury));

    // the entire TJR allocation is now fully vested, pre-cliff
    assertEq(
        wallet.vestedAmount(address(tjr), uint64(block.timestamp)),
        1_000_000e18
    );
    // and the clawback is permanently gone
    vm.prank(admin);
    vm.expectRevert(TajirVestingWallet.AlreadyRevoked.selector);
    wallet.withdrawUnreleased(address(tjr), payable(treasury));
}
```

HIGH · ISSUE 2

A single global revoked flag voids the vesting schedule for every asset held *(continued)*

Resolved

Recommendation

Make revocation per-asset and assert that value actually moved.

```
- bool revoked;
+ mapping(address => bool) revoked;   // address(0) key == native
```

```
function _vestingSchedule(uint256 totalAllocation, uint64 timestamp)
    internal view override returns (uint256)
{
    // pass the asset being queried through, and gate on revoked[asset]
}
```

Because OpenZeppelin's `_vestingSchedule` signature does not carry the asset, this requires overriding `vestedAmount(address,uint64)` and `vestedAmount(uint64)` to consult the per-asset flag, or tracking the asset in transient storage for the duration of the call.

Additionally, assert a real balance delta so the flag cannot be flipped without value moving:

```
uint256 balBefore = native ? address(this).balance
                           : IERC20(token).balanceOf(address(this));
// ... perform transfer ...
uint256 balAfter  = native ? address(this).balance
                           : IERC20(token).balanceOf(address(this));
if (balBefore - balAfter == 0) revert NothingToWithdraw();
```

Medium Severity Issues

MEDIUM · ISSUE 3

renounceOwnership() and transferOwnership() are inherited un-overridden

Resolved

Path
`src/TajirVestingWallet.sol`

Function Name
`renounceOwnership()` , `transferOwnership(address)` (both inherited from `OwnableUpgradeable`)

Description

`TajirVestingWallet` inherits `OwnableUpgradeable` through `VestingWalletUpgradeable`, where `owner()` **is** the vesting beneficiary every `release()` pays out to `owner()`. Neither `renounceOwnership()` nor `transferOwnership()` is overridden anywhere in the in-scope contracts.

The beneficiary can therefore call `renounceOwnership()`, setting `owner()` to `address(0)`. Every subsequent `release()` then attempts to pay `address(0)`: native transfers burn the value, and most ERC-20 implementations revert, permanently bricking the release path. The vesting wallet has no admin function to reassign the beneficiary, so this is unrecoverable.

`transferOwnership()` likewise allows the entire live vesting position including all future releases to be handed to an arbitrary address in a single unrestricted call, with no two-step accept and no admin oversight.

Impact High

Permanent loss or misdirection of the entire vesting allocation.

Likelihood Low

Requires the beneficiary to act against their own interest, or to be compromised. There is no attacker-controlled path.

MEDIUM · ISSUE 3

renounceOwnership() and transferOwnership() are inherited un-overridden *(continued)*

Resolved

POC

```
function test_beneficiaryCanStrandAllAssets() public {
    vm.prank(beneficiary);
    wallet.renounceOwnership();

    assertEq(wallet.owner(), address(0));

    vm.warp(block.timestamp + 400 days);      // fully vested
    vm.expectRevert();                        // ERC20 transfer to address(0)
    wallet.release(address(tjr));
}
```

Recommendation

Override both functions to revert, since neither has a legitimate use for a vesting wallet:

```
function renounceOwnership() public pure override {
    revert("TajirVestingWallet: beneficiary cannot be renounced");
}

```

```
function transferOwnership(address) public pure override {
    revert("TajirVestingWallet: beneficiary is immutable");
}
```

If beneficiary transfer is a required feature, implement it as a two-step propose/accept flow gated on `admin` approval rather than the inherited single-call form.

Medium Severity Issues

MEDIUM · ISSUE 4

Unguarded subtraction underflows against non-conserving tokens (rebasing or fee-on-transfer)

Resolved

Path
`src/TajirVestingWallet.sol`

Function Name
`withdrawUnreleased(address,address payable)`, `releasable(address)` (inherited)

Description

`totalAllocation` is recomputed **live** on every call as `balanceOf(this) + released(token)`, while `released(token)` is a **monotonically increasing** accumulator. The subtraction is unguarded:

```
1  uint256 vested = native ? vestedAmount(uint64(block.timestamp))
2                          : vestedAmount(token, uint64(block.timestamp));
3  uint256 alreadyReleased = native ? released() : released(token);
4
5  uint256 claimable = vested - alreadyReleased;      // L131  unguarded
```

For a conserving ERC-20 the two move in lockstep and the difference is always safe the equivalent fuzz property against `TajirToken` **passed**, confirming the token itself is not the problem.

However, `release(address token)` and `withdrawUnreleased(address token, ...)` both accept an **arbitrary caller-supplied token address**. A rebasing token, a fee-on-transfer token, or any asset whose balance can decrease independently will drive `balanceOf(this)` below the already-accumulated `released(token)`. `vested` then falls below `alreadyReleased` and the subtraction panics with `0x11` (arithmetic underflow).

The same expression exists in OpenZeppelin's inherited `releasable(address)`, so the panic bricks both the clawback path and the beneficiary's normal `release()` path for that asset.

Impact Medium

Permanent denial of service on the affected asset neither the beneficiary nor the admin can move it. Core protocol functionality is disabled for that token. Assets already correctly accounted for are not lost, but become unreachable.

Likelihood Medium

Requires a non-conserving asset to enter the wallet. Since anyone can transfer any ERC-20 to the wallet address and the contract imposes no allowlist, this is externally triggerable by an unprivileged party.

MEDIUM · ISSUE 4

Unguarded subtraction underflows against non-conserving tokens (rebasing or fee-on-transfer) *(continued)*

Resolved

POC

```
function test_rebasingTokenBricksReleasePath() public {
    RebasingToken rt = new RebasingToken();
    rt.mint(address(wallet), 1000e18);

    vm.warp(block.timestamp + 400 days);       // fully vested
    wallet.release(address(rt));               // released(rt) accumulates

    rt.rebaseDown(50);                         // balance halves

    vm.expectRevert();                         // panic 0x11  underflow
    wallet.releasable(address(rt));
}
```

Recommendation

Clamp the subtraction so a reduced balance degrades gracefully instead of reverting:

```
- uint256 claimable = vested - alreadyReleased;
+ uint256 claimable = vested > alreadyReleased ? vested - alreadyReleased : 0;
```

Additionally, restrict the accepted asset set to an explicit allowlist rather than any caller-supplied address. Balance-derived vesting accounting is only sound for conserving, non-rebasing, non-fee-on-transfer tokens; that assumption should be enforced in code rather than left implicit.

Medium Severity Issues

MEDIUM · ISSUE 5

initialize() never validates startTimestamp

Resolved

Path
src/TajirVestingWallet.sol

Function Name
`initialize(...)` (L82-111)

Description

The initializer validates every schedule parameter **except** `startTimestamp`:

```
1  if (admin_ == address(0)) revert ZeroAddress();            // L92
2  if (upgrader_ == address(0)) revert ZeroAddress();         // L93
3  if (trancheDuration_ == 0) revert ZeroTrancheDuration();   // L94
4  if (totalTranches_ == 0) revert ZeroTotalTranches();       // L95
5  if ((totalTranches_ * trancheDuration_) + cliffSeconds != durationSeconds) {
6      revert InvalidScheduleParams();                        // L96-101
7  }
```

The coherence check enforces that the schedule is arithmetically **consistent**, but never that it still **locks anything**. A `startTimestamp` sufficiently far in the past places `cliff()` and `end()` both behind `block.timestamp`, so `_vestingSchedule` returns `totalAllocation` on the very first call and the wallet is 100% vested at deployment.

Impact Medium

A vesting wallet that enforces no lockup while presenting as a vesting contract. The cliff and tranche schedule are silently void; nothing on-chain distinguishes this from a correctly configured wallet.

Likelihood Medium

Not reachable through the current deploy script, but reachable through the unprotected initializer through a future deployment path, or through operator error when back-dating a schedule.

MEDIUM · ISSUE 5

initialize() never validates startTimestamp *(continued)*

Resolved

POC

```
function test_pastStartTimestampVoidsLockup() public {
    uint64 pastStart = uint64(block.timestamp - 400 days);
    bytes memory init = abi.encodeCall(OriginalTajirVestingWallet.initialize,
        (beneficiary, pastStart, 360 days, 30 days, 30 days, 11, admin, upgrader));
    OriginalTajirVestingWallet w = OriginalTajirVestingWallet(payable(address(new ERC1967Proxy(address(impl), init))));

    tjr.mint(address(w), 1_000_000e18);

    // initialisation succeeded, yet the wallet is fully vested immediately
    assertEq(w.vestedAmount(address(tjr), uint64(block.timestamp)), 1_000_000e18);
    assertEq(w.releasable(address(tjr)), 1_000_000e18);
}
```

Recommendation

Require the schedule to lock value at the moment of initialization:

```
if (startTimestamp + cliffSeconds <= block.timestamp) {
    revert InvalidScheduleParams();
}
```

If back-dated schedules are an intended feature, bound how far into the past `startTimestamp` may be set and emit an event recording the deviation.

Medium Severity Issues

MEDIUM · ISSUE 6

Clawback commits an irreversible flag with no balance-delta assertion

Resolved

Path
`src/TajirVestingWallet.sol`

Function Name
`withdrawUnreleased(address,address payable)`

Description

`withdrawUnreleased` derives the amount to sweep from two independent, uncached `balanceOf` reads against an **unvalidated** token contract, commits the irreversible global flag, emits an event asserting the amount, and only then performs the transfer:

```
1  uint256 currentBalance = native ? address(this).balance
2                                  : IERC20(token).balanceOf(address(this));   // L127
3  uint256 vested = ...;                                                       // L128
4  uint256 alreadyReleased = ...;                                              // L129
5  uint256 claimable = vested - alreadyReleased;                               // L131
6  if (currentBalance <= claimable) revert NothingToWithdraw();                // L133
7  uint256 withdrawable = currentBalance - claimable;                          // L135
8  $.revoked = true;                                                           // L137
9  emit VestingRevoked(token, withdrawable);                                   // L139  before transfer
10 if (native) Address.sendValue(to, withdrawable);                            // L141
11 else SafeERC20.safeTransfer(IERC20(token), to, withdrawable);               // L142
```

Note that `vestedAmount(token, ...)` performs a **second** `balanceOf` read internally. A token that returns different values across the two reads produces an inconsistent `withdrawable`.

Nothing reconciles the asserted amount against what was actually transferred. `VestingRevoked` is emitted with a value that may not match reality, which corrupts off-chain accounting and indexers that consume it.

Impact Medium

Event data may not reflect the actual transfer, breaking downstream accounting. Combined with the missing delta assertion, the irreversible flag can be consumed while less value moves than reported.

Likelihood Medium

Requires a non-standard or adversarial token to be passed by the admin.

MEDIUM · ISSUE 6

Clawback commits an irreversible flag with no balance-delta assertion *(continued)*

Resolved

POC

```
function test_eventAssertsUntransferredAmount() public {
    InconsistentToken it = new InconsistentToken();  // balanceOf varies per call
    it.mint(address(wallet), 1000e18);

    vm.expectEmit(true, false, false, false);
    emit VestingRevoked(address(it), 1000e18);       // asserted

    vm.prank(admin);
    wallet.withdrawUnreleased(address(it), payable(treasury));

    assertLt(it.balanceOf(treasury), 1000e18);       // actually received less
}
```

Recommendation

Cache the balance once, assert the realised delta, and emit the event after the transfer with the actual amount:

```
uint256 balBefore = _balanceOf(token);
// ... compute withdrawable from the cached value only ...
$.revoked = true;
if (native) Address.sendValue(to, withdrawable);
else SafeERC20.safeTransfer(IERC20(token), to, withdrawable);
uint256 moved = balBefore - _balanceOf(token);
if (moved == 0) revert NothingToWithdraw();
emit VestingRevoked(token, moved);
```

Medium Severity Issues

MEDIUM · ISSUE 7

Schedule parameters are read live with no clamp

Resolved

Path
`src/TajirVestingWallet.sol`

Function Name
`_vestingSchedule(uint256,uint64)` (L170-183)

Description

`_vestingSchedule` reads `trancheDuration` and `totalTranches` **live from storage** on every computation, with no clamp and no monotonicity guarantee:

```
1  uint256 tranchesPassed = (timestamp - cliff()) / $.trancheDuration;    // L180
2  return (totalAllocation * tranchesPassed) / $.totalTranches;           // L181
```

There is no setter for either value, so they cannot change during normal operation. However, the contract is **UUPS upgradeable**, and an upgrade can rewrite them directly in the ERC-7201 namespaced storage slot. Because vested amounts are recomputed from current values rather than snapshotted, changing either parameter **retroactively alters how much is already vested** including downward, below amounts already released.

Impact High

An upgrade can retroactively reduce the vested entitlement of beneficiaries (for example, from 4,062,500 TJR to 406,250 TJR for a beneficiary at 18 of 36 tranches) and, in the code as reviewed, push vested amounts below released amounts, permanently bricking releases.

Likelihood Low

Requires an upgrade authorised by the upgrader. This assessment assumes the planned configuration in which upgrader is subject to an enforced 48-hour delay that the Veto Safe can cancel.

MEDIUM · ISSUE 7

Schedule parameters are read live with no clamp *(continued)*

Resolved

POC

```
function test_upgradeCanRetroactivelyShrinkVested() public {
    vm.warp(block.timestamp + 200 days);
    uint256 before = wallet.vestedAmount(address(tjr), uint64(block.timestamp));

    // upgrade to an implementation that writes a larger totalTranches
    vm.prank(upgrader);
    wallet.upgradeToAndCall(address(new MaliciousVestingImpl()), "");

    uint256 afterUpgrade = wallet.vestedAmount(address(tjr), uint64(block.timestamp));
    assertLt(afterUpgrade, before);   // already-vested amount shrank
}
```

Recommendation

Snapshot the schedule at initialization and treat it as immutable thereafter, or enforce monotonicity so a recomputed vested amount can never fall below `released()`:

```
uint256 computed = (totalAllocation * tranchesPassed) / $.totalTranches;
uint256 floor_ = released(token);
return computed < floor_ ? floor_ : computed;
```

Document explicitly that schedule parameters are outside the upgrade surface, and add a storage-layout regression test to the upgrade process.

Resolution Note

The fix addresses the accounting defect. Both vestedAmount overrides now clamp the result so that it can never fall below released(), so a changed schedule can no longer underflow and brick release() (test_R7_clamp_DOES_prevent_falling_below_released, passed on the final code).

The upgrader is also controlled with planned enforced 48-hour delay with Veto Safe cancellation.

Low Severity Issues

LOW · ISSUE 8

Tranche quantization lets a revocation strip a nearly-elapsed tranche

Acknowledged

Path
`src/TajirVestingWallet.sol`

Function Name
`_vestingSchedule(uint256,uint64)`, `withdrawUnreleased(address,address payable)`

Description

Vesting advances in discrete steps via floor division at L180 `tranchesPassed = (timestamp - cliff()) / $.trancheDuration`. Value earned in wall-clock time within an incomplete tranche is not credited until the tranche boundary is crossed.

Revocation timing is entirely unconstrained. An admin calling `withdrawUnreleased` at `cliff + k·trancheDuration - 1` sweeps a full tranche of value that the beneficiary has, in wall-clock terms, almost entirely earned.

Impact Low

The beneficiary loses up to one full tranche of value relative to a continuous-vesting interpretation. With the deployed configuration this is a bounded, single-tranche amount.

Likelihood Medium

Requires deliberate timing by the admin. No technical barrier exists.

Recommendation

Consider pro-rata vesting **within** a tranche, or introduce a notice period so revocation takes effect only at the next tranche boundary. If discrete tranches are an intentional design decision, document the timing exposure explicitly for beneficiaries.

Tajir Team's Comment

The tranche-based schedule is an intentional design decision.

QuillAudits Note

Acknowledgement does not remove the behaviour described above: a revocation executed shortly before a tranche boundary still removes that nearly-elapsed tranche from the beneficiary's entitlement.

Low Severity Issues

LOW · ISSUE 9

TajirToken.initialize silently rescales initialSupply

Acknowledged

Path
`src/token/TajirToken.sol`

Function Name
`initialize(string,string,uint256,address,address)` (L37-56)

Description

The zero-check at L46 validates the **raw** argument, while L55 mints a value scaled by 18 decimals:

```
1  if (initialSupply == 0) revert Tajir__ZeroSupply();     // L46  checks raw value
2  ...
3  _mint(initialHolder, initialSupply * 10 ** decimals());
```

The parameter is documented as *"whole-token initial supply (decimals applied internally)"* at L34, so the behaviour is intentional. However, a caller who passes an already-scaled value a very common convention, silently deploys a supply 10^18 times larger than intended. There is no upper bound to catch this.

Impact Low

A misconfigured genesis supply. The error is severe if it occurs, but immediately visible on-chain post-deployment and correctable only by redeployment.

Likelihood Low

Requires deployment-time operator error; the NatSpec does document the expected unit.

Recommendation

Add a sanity bound on the raw argument, for example `if (initialSupply > 1e12) revert Tajir__ZeroSupply();`, or accept the pre-scaled value directly and rename the parameter to `initialSupplyWei` to eliminate the ambiguity.

Tajir Team's Comment

The genesis supply of 650,000,000 (six hundred fifty million) TJR was minted at deployment and can be verified on-chain. The initializer has been consumed on the live proxy and cannot be called again.

LOW · ISSUE 9

TajirToken.initialize silently rescales initialSupply *(continued)*

Acknowledged

Reassessment

We verified on the Ethereum mainnet proxy that totalSupply() is 650,000,000 × 10^18, that Initialized(1) was emitted in the proxy's deployment transaction (`0x5db1cbe3af8cce70956d3471cfc49ee444bc9347159dbf7d9a27c67e35d3c234`), and that a further initialize call reverts. The deployment-time concern therefore did not materialise, and the consumed initializer presents no continuing live-token risk. The scaling logic is unchanged in the reviewed code, so the source-level concern remains Acknowledged rather than Resolved.

Informational Severity Issues

INFORMATIONAL · ISSUE 10

revoked has no getter and its guard precedes the authorization gate

Resolved

Path
`src/TajirVestingWallet.sol`

Function Name
`withdrawUnreleased(address,address payable)`

Description

`trancheDuration()`, `totalTranches()`, and `upgrader()` all have public getters (L148-164), but `revoked` the single most consequential state variable in the contract, and terminal once set has none. Off-chain consumers cannot determine whether a wallet has been revoked without replaying `VestingRevoked` events.

Separately, the ordering at L121-122 checks `AlreadyRevoked` **before** `NotAdmin`, so an unauthorized caller can distinguish revoked from non-revoked wallets by the revert reason.

Impact

No security impact. Observability and information-disclosure hygiene.

Recommendation

Add `function revoked() external view returns (bool)` and swap the two checks so authorization is evaluated first.

Informational Severity Issues

INFORMATIONAL · ISSUE 11

withdrawUnreleased re-derives releasable() inline

Resolved

Path
`src/TajirVestingWallet.sol`

Function Name
`withdrawUnreleased(address,address payable)` (L127-131)

Description

L127-131 recomputes `vested - alreadyReleased` inline rather than calling the inherited `releasable(token)` helper, which computes exactly the same quantity. Two definitions of one concept now exist in the codebase and can drift apart under future edits a maintenance hazard that also doubles the number of `balanceOf` calls to an untrusted token.

Impact

No security impact today. Maintainability and gas.

Recommendation

Call the inherited `releasable(token)` / `releasable()` helper and cache the result.

Informational Severity Issues

INFORMATIONAL · ISSUE 12

Floor division routes rounding remainders away from the beneficiary

Resolved

Path
`src/TajirVestingWallet.sol`

Function Name
`_vestingSchedule(uint256,uint64)`

Description

`(totalAllocation * tranchesPassed) / $.totalTranches` truncates on every computation, so the remainder consistently favours the contract over the beneficiary until `end()` is reached, at which point the `timestamp >= end()` branch returns the full allocation and settles the difference.

Small allocations follow the same floor division: a 1-wei allocation vests zero until end() and is released in full only through the terminal branch, while other allocations smaller than totalTranches can become partly vested before end() (for example, 10 wei with 11 tranches vests 9 wei after the tenth tranche).

Impact

No loss at maturity. Dust-scale under-crediting mid-schedule.

Recommendation

Acceptable as-is given the terminal branch settles exactly. Document the rounding direction.

Informational Severity Issues

INFORMATIONAL · ISSUE 13

upgradeToAndCall inherited payable with no ETH-handling path

Resolved

Path
`src/token/TajirToken.sol`

Function Name
`upgradeToAndCall(address,bytes)` (inherited from `UUPSUpgradeable`)

Description

`TajirToken` inherits `UUPSUpgradeable.upgradeToAndCall`, which is `payable`, but declares no `receive()`, no `fallback()`, and no sweep function. Any ETH sent with an upgrade call is permanently locked in the contract.

Impact

Unrecoverable loss of any ETH sent during an upgrade. Zero under correct usage.

Recommendation

Override `upgradeToAndCall` to reject non-zero `msg.value`, or add an admin-gated ETH sweep.

Tajir Team's Comment

Addressed in the Review 4 code: _authorizeUpgrade always reverts, so upgradeToAndCall cannot succeed and ETH cannot be retained through a successful upgrade call.

Reassessment

Verified at commit `94551139e0cb1f03f97bdecf1344003331f7182a` : every upgradeToAndCall call reverts, including calls that carry msg.value, so no ETH can be retained through a successful upgrade call. The Review 4 implementation was applied to the live proxy in transaction `0xbb10e7ba9158d2582e76c21627416290722b07348bc00c86dd69265f238fcdbf` (block 26,204,693, 16 October 2026 11:07:47 UTC). Post-upgrade checks on the live chain confirm that upgradeToAndCall now reverts for the UPGRADER_ROLE holder (the project's 3-of-5 Safe, which is not behind a timelock) and for any other caller, including calls that carry ETH.

This assessment concerns TajirToken only and does not state that ETH can never reach the token address by other mechanisms. The vesting wallet's own payable upgradeToAndCall is a separate matter and is not closed by the token lock.

Functional Tests

Test environment and results. Every test listed below was executed. They ran against commit `94551139e0cb1f03f97bdecf1344003331f7182a` (final TajirToken.sol, TajirTokenProxy.sol and TajirVestingWallet.sol) with Foundry forge 1.7.1, solc 0.8.27 (optimizer 200 runs, EVM cancun) and OpenZeppelin Contracts / Contracts Upgradeable v5.5.0, on a local EVM.

The project's own suite (`forge test` , excluding the QuillAudits files) passed: 1,098 tests, 0 failures.

1,098 **0 failures**
Project suite (`forge test` , excluding QuillAudits files)

69 **0 failures**
QuillAudits suites: tests and invariants (`test/unit/QA*`)

The QuillAudits suites passed with `forge test --match-path "test/unit/QA*"` : 69 tests and invariants, 0 failures. The token items are covered by QATokenReview4.t.sol and QAFunctionalChecklist.t.sol; the vesting items by QAFunctionalChecklist.t.sol, together with the project's TajirVestingWallet.t.sol and TajirVestingWalletAudit.t.sol.

Initialization

- ✔ Should initialize with valid parameters and set beneficiary, admin, upgrader, trancheDuration and totalTranches correctly
- ✔ Should revert when `admin_` is the zero address
- ✔ Should revert when `upgrader_` is the zero address
- ✔ Should revert when `trancheDuration_` is zero
- ✔ Should revert when `totalTranches_` is zero
- ✔ Should revert when `(totalTranches * trancheDuration) + cliffSeconds != durationSeconds`
- ✔ Should revert on a second call to the 9-argument `initialize`
- ✔ Should revert on the inherited 3-argument `initialize` after the 9-argument path has run
- ✔ Should **not** allow the inherited 3-argument `initialize` to be called on an uninitialized proxy
- ✔ Should revert when `startTimestamp + durationSeconds` is not in the future (`ScheduleLocksNothing`)
- ✔ Should confirm the implementation contract itself is permanently disabled by `_disableInitializers()`

Vesting schedule

- ✔ Should return zero vested before `cliff()` for both native and ERC-20 tracks
- ✔ Should return zero vested at exactly `cliff() - 1`
- ✔ Should vest exactly one tranche at `cliff() + trancheDuration`
- ✔ Should vest exactly `k` tranches at `cliff() + k·trancheDuration`

- Should not advance vesting between tranche boundaries
- Should return the full allocation at `end()`
- Should return the full allocation for any timestamp beyond `end()`
- Should handle `totalTranches == 1` correctly
- Should floor each tranche amount, so that a 1-wei balance vests zero until end() and the remainder settles at `end()`

Release

- Should release the vested amount to the beneficiary for ERC-20
- Should release the vested amount to the beneficiary for native
- Should be callable by any address while still paying `owner()`
- Should release nothing when called twice in the same tranche window
- Should accumulate `released(token)` monotonically across releases
- Should revert immediately on `renounceOwnership()` and `transferOwnership()` with `OwnershipLocked`
- Should not revert for a conserving token at any point in the schedule

Clawback / revocation

- Should allow `admin` to withdraw the unvested surplus
- Should revert with `NotAdmin` for any non-admin caller
- Should revert with `ZeroAddress` when `to` is the zero address
- Should revert with `NothingToWithdraw` when `currentBalance <= claimable`
- Should revert with `AlreadyRevoked` on a second call
- Should leave the claimable (vested but unreleased) portion in the contract
- Should **not** fully vest other assets when one asset is revoked
- Should **not** allow the flag to be set without a non-zero balance delta
- Should emit `VestingRevoked` with the amount actually transferred
- Should handle native revocation via `address(0)` correctly

Access control and upgradeability

- Should allow only `upgrader` to authorize an upgrade on the vesting wallet
- Should revert with `NotUpgrader` for any other caller
- Should preserve ERC-7201 namespaced storage across an upgrade
- Should not allow an upgrade to reduce vested below `released()`
- Should revert any `TajirToken` upgrade by the `UPGRADER_ROLE` holder (`Not Upgradeable Anymore!`)

- Should revert any `TajirToken` upgrade by an unauthorised caller (`AccessControlUnauthorizedAccount`) and by a newly granted `UPGRADER_ROLE` holder
- Should revert a `TajirToken` upgrade carrying initialization calldata or `msg.value` , retaining no ETH
- Should revert `upgradeToAndCall` called directly on the `TajirToken` implementation (`UUPSUnauthorizedCallContext`)
- Should confirm that no other callable `TajirToken` function can change the implementation (ABI enumerated)
- Should allow the last `DEFAULT_ADMIN_ROLE` holder to renounce (plain `AccessControl`); after Review 4 this cannot affect supply or upgrades

TajirToken

- Should mint exactly `initialSupply * 10 ** decimals()` to `initialHolder`
- Should revert when `initialHolder` is the zero address
- Should revert when `upgrader` is the zero address
- Should revert when `initialSupply` is zero
- Should have no function that mints after initialization
- Should reduce the holder's balance and `totalSupply` on holder self-burn via `burn(uint256)`
- Should reduce balance and `totalSupply` and consume the allowance on `burnFrom(address,uint256)` by an approved spender, paying nothing to the spender
- Should not decrement an unlimited allowance on `burnFrom`
- Should revert `burn` above balance (`ERC20InsufficientBalance`) and `burnFrom` above allowance (`ERC20InsufficientAllowance`)
- Should provide no privileged burn: the role holder cannot burn another holder's tokens without an allowance
- Should preserve storage layout between Review 3 (c6c8603) and Review 4 (94551139): both use ERC-7201 namespaced storage only, with an identical ABI
- Should preserve name, symbol, decimals, total supply, balances, allowances, permit nonces, domain separator and roles when a Review 3 proxy is upgraded to the Review 4 implementation, after which further upgrades and re-initialization revert
- Should support EIP-2612 `permit` with a valid signature
- Should reject a replayed `permit` signature
- Should reject a `permit` signature past its deadline
- Should produce a domain separator bound to the correct chainId

Non-standard token handling

- Should not brick `releasable()` when a rebasing token's balance decreases
- Should not brick `releasable()` for a fee-on-transfer token

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates **everything the protocol does not fully control**.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
VestingWallet Upgradeable (OpenZeppelin)	Base Contract	Provides `released` / `releasable` / `release`, the `vestedAmount` accumulators, and `owner()` as beneficiary	Correct linear-vesting accounting; initializer semantics as documented	The inherited 3-argument initializer is overridden to revert in the final code (previously exposed, Issue 1). Separate native and per-ERC-20 accumulators remain; only the wallet's single `vestedAsset` reaches the schedule
VestingWalletCliff Upgradeable (OpenZeppelin)	Base Contract	Provides `cliff()` and the pre-cliff zero branch	Cliff is enforced before any other schedule logic	Cliff ordering relative to the `revoked` branch was verified correct by fuzzing
Ownable Upgradeable (OpenZeppelin)	Base Contract	`owner()` **is** the vesting beneficiary; payout target for every release	Owner is the intended beneficiary	`renounceOwnership` / `transferOwnership` revert in the final code (previously callable, Issue 3); the beneficiary is fixed at initialization
UUPS Upgradeable (OpenZeppelin)	Base Contract	Upgrade mechanism for the vesting wallet; inherited by TajirToken	`_authorizeUpgrade` correctly gates the upgrade path	TajirToken: _authorizeUpgrade always reverts; the Review 4 implementation was applied to the live proxy on 16 October 2026 (transaction `0xbb10e7ba9158d2582e76c21627416290722b07348bc00c86dd69265f238fcdbf`), so the live token has no working upgrade path. TajirVestingWallet: upgradeable by upgrader; an upgrade can rewrite schedule logic; the inherited upgradeToAndCall is payable
AccessControl Upgradeable (OpenZeppelin)	Base Contract	Role management for TajirToken (`DEFAULT_ADMIN_ROLE`, `UPGRADER_ROLE`)	Role admin relationships behave as documented;	Both roles are held by the project's 3-of-5 Safe `0xBA1D51BBB17Fd24Fd6bF2c030F3ca0E3a3B4d22E`. The Safe is not behind a timelock; multisig control is not a delay.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
AccessControl Upgradeable (OpenZeppelin) *(continued)*			`DEFAULT_ADMIN_ROLE` is self-administering	Since the Review 4 upgrade (16 October 2026), the Safe can only grant and revoke roles; UPGRADER_ROLE authorises nothing, and upgrade attempts by the Safe revert on the live chain. renounceRole can remove the last admin
ERC1967Proxy (OpenZeppelin)	Base Contract	Proxy implementation for `TajirTokenProxy` and the vesting wallets	Standard ERC-1967 slot handling and atomic init when `data` is supplied	The vesting deployment script now passes init data to the proxy constructor (atomic; previously empty init data, Issue 1), and the live token proxy was deployed and initialized in one transaction. The 9-argument vesting `initialize` remains unrestricted, so atomic initialization is enforced by the deployment script. The token proxy remains in place after Review 4; only upgrade authorisation is disabled. The wrapper drops `payable`
ERC20Permit Upgradeable (OpenZeppelin)	Base Contract	EIP-2612 gasless approvals for TajirToken	Correct EIP-712 domain construction, nonce handling, and `ecrecover` usage	Standard OpenZeppelin implementation; no deviation found in this audit
SafeERC20 (OpenZeppelin)	Library	Wraps ERC-20 transfers in the clawback path (`safeTransfer`)	Correctly handles non-standard return values and reverts on failure	Does **not** protect against fee-on-transfer or rebasing accounting drift
Address (OpenZeppelin)	Library	`sendValue` for the native clawback path	Forwards all gas and bubbles reverts	A contract recipient with a reverting `receive()` blocks the native clawback
Caller-supplied `token` address	External ERC-20 Contract	Passed into `release(token)` and `withdrawUnreleased(token, to)`	Only the wallet's `vestedAsset` (fixed at initialization) is used for accounting; it is assumed conserving, with honest `balanceOf` and `transfer`	In the final code, arbitrary tokens can neither flip revocation (`NotVestedAsset`) nor brick releases (clamped); previously possible (Issues 2, 4). Residual assumptions: assets other than `vestedAsset` cannot be withdrawn; a non-conserving vested asset could latch revocation with a minimal transfer;

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Caller-supplied `token` address *(continued)*				`vestedAsset` must be a deployed contract
Tajir Chain (Chain ID 3377)	Blockchain / Execution Environment	Network where TJR is the **native** gas asset; native-asset vesting wallets would be deployed here	Native asset is TJR	A wrapped native token (WTJR) exists on this chain (evidence in the next row), so the earlier assumption that no ERC-20 representation of TJR exists no longer holds. Chain ID 7733 is the Tajir testnet. The planned main-allocation vesting proxies are on Ethereum mainnet and hold ERC-20 TJR; this row does not apply to them
Wrapped native TJR (WTJR)	External ERC-20 Contract (Tajir Chain)	Not used by the in-scope contracts; relevant only if WTJR is sent to a wallet that vests native TJR	Described by the development team as a standard OP Stack native-token wrapper. On-chain evidence (Tajir Chain mainnet, Chain ID 3377, read at block 2,808,513): the contract at the OP Stack predeploy address `0x4200000000000000000000000000000000000006` returns name "Wrapped Tajir", symbol "WTJR", decimals 18, version "1.1.1" and total supply 0. Its 2,846-byte runtime code (code hash `0xe354d52f6267708b1b69faf84795aaf6abfa01e623ca8f912023399888e58fdb`) is present unchanged from the genesis block and is not behind an EIP-1967 proxy, which identifies it as a genesis predeploy.	**Vesting exploitability (conclusion):** the previously described dual-accumulator attack is **not reachable** in the final vesting implementation (TajirVestingWallet.sol at commit `b04f613864d463d88963630c25fbde68187cb9ca`; logic identical at `9455113 9e0cb1f03f97bdecf1344003331f7182a`). Supporting test: QAWrappedNativeDualAccumulator.t.sol (forge 1.7.1, `forge test --match-path test/unit/QAWrappedNativeDualAccumulator.t.sol`), 4 tests passed: `test_historical_wrapped_revoke_fully_vests_native_track` reproduces the attack against the pre-fix code, and `test_final_wrapped_value_cannot_accelerate_or_duplicate_native_track` shows it is blocked in the final code.

2. Entry & Exit Points Analysis (Function-Level)

This is the **core of the threat model**. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Contract Name	Function	Category
TajirVestingWallet.sol	`constructor()`	**What this function can do** Disable initializers on the implementation contract **What this function cannot/should not do** Initialize any proxy state; protect the proxy from being initialized by a third party **Main invariant(s)** The implementation contract can never be initialized directly **Access level** Implicit (once at deployment)
TajirVestingWallet.sol	`initialize(address,uint64,uint64,uint64,uint64,address,address,address)`	**What this function can do** Set beneficiary (as `owner()`), start, duration, cliff, tranche schedule, admin, upgrader and the single `vestedAsset`, exactly once **What this function cannot/should not do** Be callable twice; accept an incoherent schedule or one whose `end()` has already passed **Main invariant(s)** `(totalTranches × trancheDuration) + cliffSeconds == durationSeconds`; `start + duration > now`; runs exactly once **Access level** Unrestricted on an uninitialized proxy; protection relies on atomic deployment (initialization data passed to the proxy constructor)
TajirVestingWallet.sol	`initialize(address,uint64,uint64)` *(inherited, overridden)*	**What this function can do** Nothing; it always reverts **Main invariant(s)** The 3-argument path cannot initialize a wallet **Access level** Any caller (reverts)

Contract Name	Function	Category
TajirVesting Wallet.sol	`withdrawUnreleased(address,address payable)`	**What this function can do** Claw back the unvested surplus of the wallet's single `vestedAsset` to a recipient, once, and latch the revoke flag **What this function cannot/should not do** Touch the claimable (vested-but-unreleased) portion; accept any other asset (`NotVestedAsset`); latch the flag without moving value (`NoBalanceMoved`); send to the zero address; be called twice **Main invariant(s)** Claimable portion remains in the contract; the flag is one-shot and applies to the vested asset only **Access level** `admin` only (checked first)
TajirVesting Wallet.sol	`release()` / `release(address)` *(inherited)*	**What this function can do** Transfer the vested-but-unreleased amount of native or an ERC-20 to `owner()` **What this function cannot/should not do** Pay any address other than `owner()` ; release more than vested; revert for a conserving token **Main invariant(s)** `released ≤ vested` ; `released` is monotonically non-decreasing **Access level** Permissionless (funds always route to `owner()`)
TajirVesting Wallet.sol	`renounceOwnership()` / `transferOwnership(address)`	**What this function can do** Nothing; both revert with `OwnershipLocked` **Main invariant(s)** The beneficiary is fixed for the wallet's lifetime **Access level** Any caller (reverts)
TajirVesting Wallet.sol	`upgradeToAndCall(address,bytes)` *(inherited UUPS)*	**What this function can do** Replace the vesting-wallet implementation, including logic governing the vesting schedule. The vesting wallet remains upgradeable; the TajirToken upgrade lock does not apply to it

Contract Name	Function	Category
TajirVesting Wallet.sol *(continued)*	`upgradeToAndCall(address,bytes)` *(inherited UUPS)* *(continued)*	**What this function cannot/should not do** Alter already-vested entitlements retroactively; break ERC-7201 storage layout **Main invariant(s)** Vested amounts never decrease below released() (the clamp fixes the accounting underflow) (see Issue 7) **Access level** upgrader only, via `_authorizeUpgrade` (NotUpgrader for any other caller). Governance assumption: upgrader is planned to sit behind an enforced 48-hour delay that the Veto Safe can cancel; this is not yet deployed or verified (see Vesting Deployment and Governance)
TajirVesting Wallet.sol	`hasRevoked()` / `vestedAsset()` / `trancheDuration()` / `totalTranches()` / `upgrader()`	**What this function can do** Expose revocation state, the vested asset, and schedule and role configuration for off-chain verification **What this function cannot/should not do** Mutate state **Main invariant(s)** N/A (view) **Access level** Public view
TajirVesting Wallet.sol	`receive()` *(inherited)*	**What this function can do** Accept native value; for a wallet vesting native TJR this increases the allocation, while for a wallet vesting ERC-20 TJR the value sits outside the schedule and cannot be withdrawn **What this function cannot/should not do** Accept value in a way that inflates an already-revoked allocation **Main invariant(s)** Post-revocation deposits of the vested asset are releasable on arrival **Access level** Permissionless

Contract Name	Function	Category
TajirToken.sol	`constructor()`	**What this function can do** Disable initializers on the implementation **What this function cannot/should not do** Initialize proxy state **Main invariant(s)** Implementation is never directly initializable **Access level** Implicit (once at deployment)
TajirToken.sol	`initialize(string, string,uint256, address,address)`	**What this function can do** Set name/symbol, mint the 650,000,000 TJR initial supply to `initialHolder`, grant `DEFAULT_ADMIN_ROLE` and `UPGRADER_ROLE` **What this function cannot/should not do** Run twice; mint again afterwards **Main invariant(s)** Single mint at initialization; no further minting **Access level** Consumed on the live proxy (deployed and initialized atomically; re-initialization reverts)
TajirToken.sol	`upgradeToAndCall( address,bytes) / _authorizeUpgrade( address)`	**What this function can do** Nothing: upgrade attempts always revert, including those from UPGRADER_ROLE (`Not Upgradeable Anymore!`); any other caller reverts with AccessControlUnauthorizedAccount. The live token implementation is fixed at `0xa1F5F61Ae9C55b16Cf9FeAC6b499b00C563de6c5` following the recorded Review 4 upgrade (transaction `0xbb10e7ba9158d2582e76c21627416290722b07348bc00c86dd69265f238fcdbf`, 16 October 2026) **What this function cannot/should not do** Change the token implementation by any route **Main invariant(s)** The token implementation can never change **Access level** No working upgrade path for any caller, including the UPGRADER_ROLE holder (3-of-5 Safe)

Contract Name	Function	Category
TajirToken.sol	`grantRole / revokeRole / renounceRole / hasRole / getRoleAdmin` *(inherited)*	**What this function can do** Manage `DEFAULT_ADMIN_ROLE` and `UPGRADER_ROLE` **What this function cannot/should not do** Mint, pause, blacklist or burn; none of these capabilities exists **Main invariant(s)** `DEFAULT_ADMIN_ROLE` administers both roles; the last admin can renounce **Access level** `DEFAULT_ADMIN_ROLE` for grant/revoke; the role holder for renounce. In the final implementation the remaining role powers are role management only: no further minting, no pause, no blacklist, no working upgrade path and no privileged administrative burn
TajirToken.sol	`burn(uint256)`	**What this function can do** Destroy the caller's own tokens, reducing `totalSupply` **What this function cannot/should not do** Burn another account's tokens **Main invariant(s)** Balance and total supply decrease by the burned amount **Access level** Any holder
TajirToken.sol	`burnFrom(address, uint256)`	**What this function can do** Destroy tokens from an account that granted the caller an allowance, consuming the allowance and reducing `totalSupply` **What this function cannot/should not do** Burn without sufficient allowance; no role can bypass the allowance **Main invariant(s)** Balance, total supply and allowance decrease by the burned amount (an unlimited allowance is not decremented) **Access level** Approved spender

Contract Name	Function	Category
TajirToken.sol	`permit(...)` / `nonces(address)` / `DOMAIN_SEPARATOR()` *(inherited)*	**What this function can do** Approve by EIP-712 signature without an on-chain transaction from the owner **What this function cannot/should not do** Permit signature replay, cross-chain replay, or use past the deadline **Main invariant(s)** Each nonce is consumed exactly once; domain separator binds to chainId **Access level** Permissionless with a valid signature
TajirToken.sol	`transfer` / `transferFrom` / `approve` *(inherited ERC20)*	**What this function can do** Standard ERC-20 value movement and allowance management **What this function cannot/should not do** Deviate from ERC-20 semantics; apply fees or rebase **Main invariant(s)** Sum of balances equals total supply; total supply equals the initial supply minus burned tokens, with no further minting **Access level** Permissionless
TajirToken Proxy.sol	`constructor(address,bytes)`	**What this function can do** Deploy an ERC-1967 proxy pointing at an implementation, optionally executing init calldata atomically **What this function cannot/should not do** Be deployed with empty `data` when the implementation has an unprotected initializer **Main invariant(s)** Implementation slot set exactly once at construction **Access level** Implicit (once at deployment). Note: not `payable`, unlike the base

3. Asset Flow Mapping (Critical Contracts)

This section identifies **where money actually lives** and **how it moves**.

3.1 Asset-Holding Contracts

Contract	Asset Type	Custodied Assets
TajirVestingWallet.sol	Native (TJR on Tajir Chain, Chain ID 3377)	Vesting allocation of a wallet whose vested asset is native TJR
TajirVestingWallet.sol	ERC20	Vesting allocation of a wallet whose vested asset is ERC-20 TJR (planned main allocations on Ethereum mainnet); other tokens received sit outside the schedule
TajirTokenProxy.sol	ERC20	Holds no external value; custodies the full TajirToken supply ledger in its own storage

3.2 Asset Entry & Exit Functions

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
TajirVesting Wallet.sol	**receive()**	Native	–	Public	Native-asset wallet: increases the allocation, and post-revocation deposits are releasable on arrival. ERC-20-asset wallet: the value sits outside the schedule and cannot be withdrawn
TajirVesting Wallet.sol	*(direct ERC20 transfer in)*	ERC20	–	Public	Vested asset: increases the allocation. Any other token: sits outside the schedule and cannot be withdrawn; it can neither brick releases nor trigger revocation
TajirVesting Wallet.sol	**release()**	–	Native	Public	Native-asset wallet only; pays `owner()`, which cannot be the zero address (ownership is locked)
TajirVesting Wallet.sol	**release(address)**	–	ERC20	Public	Vested asset only (other tokens move nothing); underflow is clamped

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
TajirVesting Wallet.sol	**withdrawUnreleased(address, address payable)**	–	Native or ERC20 (vested asset only)	`admin`	One-shot and irreversible; sweeps only the unvested portion of the wallet's vested asset; requires a balance delta; does not affect any other asset; `to` must be non-zero, and there is no check against `to == address(this)`
TajirVesting Wallet.sol	**upgradeToAndCall(address, bytes)**	–	–	`upgrader`	Can rewrite schedule logic governing every future exit. Payable: ETH sent with the call adds to the vested allocation of a native-TJR wallet; on an ERC-20 wallet it cannot be withdrawn (SOW 1 finding A-01). Not affected by the token lock
TajirToken.sol	**initialize(...)**	–	ERC20 (minted)	Consumed on the live proxy	Minted the 650,000,000 TJR initial supply once; cannot run again
TajirToken.sol	**transfer / transferFrom**	ERC20	ERC20	Public	Standard ERC-20; no fee, no rebase
TajirToken.sol	**burn(uint256)**	ERC20	*(destroyed)*	Holder	Reduces the holder's balance and total supply; burned tokens are not paid to any recipient
TajirToken.sol	**burnFrom(address,uint256)**	ERC20	*(destroyed)*	Approved spender	Reduces the holder's balance and total supply and consumes the allowance; burned tokens are not paid to the spender or any recipient
TajirToken.sol	**upgradeToAndCall(address, bytes)**	–	–	`UPGRADER_ROLE`	Always reverts on the live token since the Review 4 upgrade (transaction `0xbb10e7ba9158d2582e76c21627416290722b07348bc00c86dd69265f238fcdbf`), so no ETH sent with the call can be retained

Fuzzing Results

All rows below were run on 16 October 2026 against commit `94551139e0cb1f03f97bdecf1344003331f7182a`. TajirVestingWallet logic at that commit is identical to `9b174a414604366232f8aa66737c75fcd1808b84` and `b04f613864d463d88963630c25fbde68187cb9ca`. Tools: Foundry forge 1.7.1, solc 0.8.27 (optimizer 200 runs, EVM cancun), OpenZeppelin v5.5.0.

- **Stateful invariants:** QAVestingFinalInvariants.t.sol, run with `forge test --match-path test/unit/QAVestingFinalInvariants.t.sol`, using 1,000 runs at depth 20 (20,000 calls per invariant, 0 reverts). The campaign covers an Ethereum-shape wallet vesting ERC-20 TJR and a Tajir Chain-shape wallet vesting native TJR, with top-ups, stray ERC-20, ETH and wrapped-native deposits, releases, time warps, revocations and attempted revocations of non-vested assets, on a 30-day cliff plus eleven 30-day tranches (360 days).
- **Stateless fuzz tests:** the project's fuzz tests used 256 runs each.

History. An earlier fuzzing campaign at the initial-review commit `01802d8370699dbe09ec1113df59c6388c32eb51` (July 2026) reported failures for initializedWalletHasValidatedParams, ownerIsIntendedBeneficiary, tjrScheduleUnaffectedByOtherAssetRevoke, clawbackSurvivesForUnrevokedAsset, revokeRequiresNonZeroSurplusMoved, heldAssetsAlwaysHaveARecipient and vestedNeverBelowReleased, which led to Issues 1–4. Every row in the table below was re-run on the final code, and those earlier results are superseded.

Invariants Tested	Status
releasableNeverReverts – releasable() never reverts for the vested asset or any other asset at any point in the schedule	✔ Passed
releasedNeverExceedsVested – released amount never exceeds vestedAmount(asset, now) for the vested asset of each wallet	✔ Passed
preCliffNothingReleasable – no value is releasable at any timestamp before cliff()	✔ Passed
perAssetIsolation – assets other than the wallet's vested asset (other ERC-20s, ETH on an ERC-20 wallet, wrapped native on a native wallet) are never released and never reach the beneficiary	✔ Passed
beneficiaryReceiptsMatchAccumulator – value received by owner() equals released()	✔ Passed
vestedAssetConservation – assets in equals the wallet balance plus the beneficiary's receipts plus the amount clawed back	✔ Passed
vestedBoundedByAllocation – vested amount never exceeds the wallet balance plus released	✔ Passed

Invariants Tested	Status
solvent – contract balance always covers the unreleased vested portion	✔ Passed
tokenConservation – every non-vested asset sent to a wallet remains held by it (the vested asset is covered by vestedAssetConservation)	✔ Passed
initializedWalletHasValidatedParams – an initialized wallet always has non-zero trancheDuration and totalTranches	✔ Passed
ownerIsIntendedBeneficiary – owner() equals the beneficiary supplied by the deployer	✔ Passed
tjrScheduleUnaffectedByOtherAssetRevoke – the vested asset follows the cliff and tranche formula unless that asset itself is revoked; activity in other assets, including attempted revocations, never changes it	✔ Passed
clawbackSurvivesForUnrevokedAsset – the clawback right survives until the vested asset itself is revoked	✔ Passed
revokeRequiresNonZeroSurplusMoved – the revocation flag cannot be latched without a non-zero balance delta	✔ Passed
Project fuzz tests – testFuzz_VestedAmount, testFuzz_WithdrawUnreleased, testFuzz_Security (256 runs each)	✔ Passed

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of Tajir Chain. We performed our audit according to the procedure described above.

13 Issues were found in Tajir Chain in Review 1, and their statuses were updated in the subsequent reviews.

Review 3 (token burn functions) found no new vulnerabilities.

Review 4 (commit `94551139e0cb1f03f97bdecf1344003331f7182a`) found no new vulnerabilities. It disables TajirToken implementation upgrades. The Review 4 implementation was applied to the live proxy on 16 October 2026 (transaction `0xbb10e7ba9158d2582e76c21627416290722b07348bc00c86dd69265f238fcdbf`) and verified on-chain. In the final reviewed code, 11 issues are Resolved and 2 are Acknowledged.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we've secured over 1400 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

October 2026

For



Tajir Chain

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com